

Reporte del Proyecto: Sistema de Gestión y Control de Eventos Académicos



Asignatura: Programación Web

Proyecto: Sistema de Gestión y Control de Eventos Académicos

Tecnologías principales: Javalin, Hibernate, H2, Flyway, React, Vite, Chart.js, html5-qrcode, Docker.

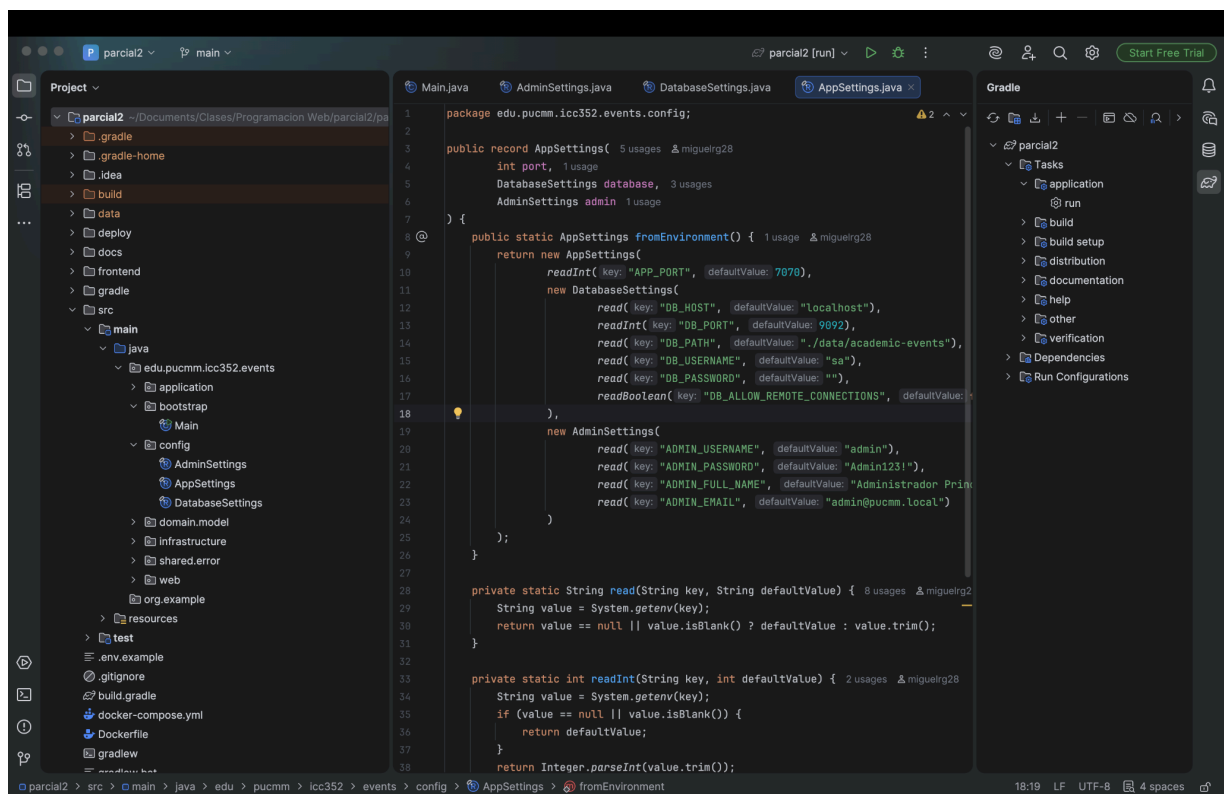
Realizado por Miguel G. Rodríguez y José A. Hidalgo

Introducción

El presente proyecto consiste en el desarrollo de una aplicación web para la gestión y control de eventos académicos dentro de una universidad. El sistema fue diseñado para cubrir las necesidades de distintos tipos de usuarios, específicamente administradores, organizadores y participantes, permitiendo manejar el ciclo completo de un evento: creación, publicación, inscripción, validación de asistencia mediante código QR y consulta de estadísticas.

La solución fue construida como una aplicación *full stack*. En el *backend* se utilizó Java con Javalin como *framework* HTTP, Hibernate como ORM, H2 en modo servidor como motor de base de datos y Flyway para la creación automática del esquema. En el *frontend* se utilizó React con Vite para ofrecer una experiencia moderna, dinámica y responsiva, apoyada en *fetch*, Chart.js y html5-qrcode para las interacciones en tiempo real.

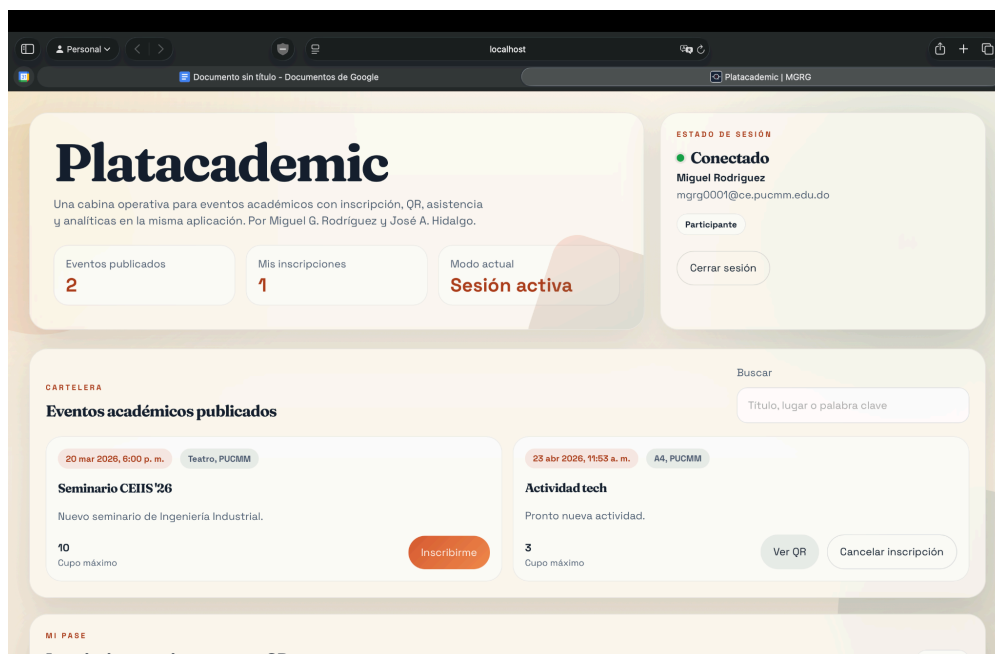
El propósito del proyecto no fue únicamente cumplir con los requerimientos funcionales del enunciado, sino también aplicar principios de organización del software, separación de responsabilidades, modularidad, persistencia desacoplada y despliegue en contenedores Docker.



1. Descripción general de la solución

La aplicación fue concebida como una plataforma centralizada para eventos académicos. Su funcionamiento se apoya en tres grandes ejes: Gestión de usuarios y roles, Gestión operativa de eventos e inscripciones, y Validación de asistencia y análisis estadístico.

El sistema ofrece diversas funcionalidades, incluyendo: la creación automática de un usuario administrador inicial al arrancar la aplicación y el inicio de sesión mediante sesión HTTP. En cuanto a la gestión de eventos, permite crear, editar, cancelar y publicar eventos, así como listar los eventos disponibles para los participantes. Para las inscripciones, el sistema permite a los usuarios inscribirse y cancelar su inscripción, incluyendo validaciones de cupo y duplicidad. Cada inscripción genera un código QR único que se utiliza para registrar la asistencia mediante escaneo. Finalmente, el sistema permite visualizar estadísticas del evento y administrar usuarios y eventos desde un módulo de moderación.



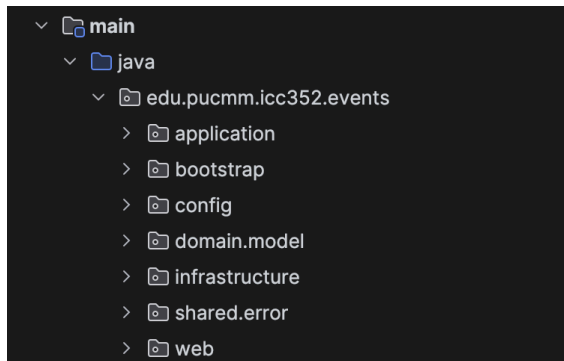
2. Arquitectura general del proyecto

El proyecto está dividido en dos capas principales: el Backend, que expone la API REST, administra la lógica del negocio, la persistencia y la seguridad, y el Frontend, que consume la API, maneja la sesión del usuario y presenta las interfaces por rol.

Adicionalmente, el proyecto incluye despliegue contenedorizado con Docker multi-stage, y un proxy inverso con Nginx para HTTPS y redirección de 80 -> 443.

La organización del backend responde a una estructura por capas que incluye:

- **domain:** entidades y reglas del negocio.
- **application:** casos de uso y servicios.
- **infrastructure:** persistencia, base de datos y QR.
- **web:** controladores HTTP, DTOs y manejo de excepciones.



- **bootstrap:** composición de dependencias y arranque de la aplicación.

El *backend* fue desarrollado con Java 25, utilizando Gradle como herramienta de construcción. Los componentes clave incluyen Javalin 7 como framework HTTP, Hibernate ORM para el mapeo objeto-relacional, H2 en modo servidor para la base de datos, y Flyway para las migraciones. Además, se utilizó BCrypt para el *hash* de contraseñas y ZXing para la generación y lectura de códigos QR. Estas dependencias se declaran en el archivo de construcción del proyecto, asegurando la compilación, ejecución y empaquetado consistente de la aplicación.

```

dependencies {
    implementation 'io.javalin:javalin:7.0.1'
    implementation 'org.hibernate.orm:hibernate-core:6.6.18.Final'
    implementation 'org.flywaydb:flyway-core:11.13.3'
    implementation 'com.h2database:h2:2.4.240'
    implementation 'org.mindrot:jbcrypt:0.4'
    implementation 'com.google.zxing:core:3.5.4'
    implementation 'com.google.zxing:javase:3.5.4'
    implementation 'com.fasterxml.jackson.datatype:jackson-datatype-jsr310:2.20.0'
    implementation 'ch.qos.logback:logback-classic:1.5.20'

    testImplementation platform('org.junit:junit-bom:5.13.4')
    testImplementation 'org.junit.jupiter:junit-jupiter'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}

```

El punto de entrada del sistema se encuentra en la clase principal del *backend*. Durante la inicialización se realiza la lectura de variables de entorno, la creación del *ObjectMapper*, el inicio del servidor H2, la ejecución de migraciones Flyway y la creación del *SessionFactory* de Hibernate. Posteriormente, se construyen los servicios y repositorios, se registran los controladores HTTP y se publica el *frontend* compilado como archivos estáticos.

La configuración se centraliza en *AppSettings*, *DatabaseSettings* y *AdminSettings*, lo que permite modificar el puerto, la base de datos y las credenciales iniciales a través de variables de entorno sin alterar el código fuente.

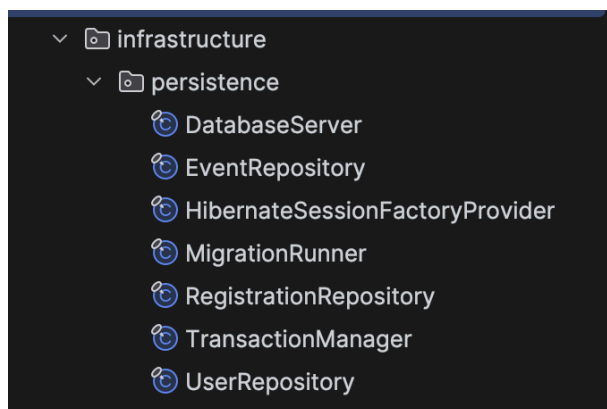
El modelo del sistema gira en torno a tres entidades principales: User, Event y Registration. Los roles contemplados son ADMIN, ORGANIZER y PARTICIPANT. Los estados principales de evento e inscripción permiten controlar el flujo del negocio.

3. Seguridad y autenticación

La autenticación del sistema se basa en sesión HTTP, cumpliendo con el requerimiento del proyecto. Al iniciar sesión, el *backend* almacena el identificador del usuario autenticado en la sesión. Cada petición protegida valida la existencia de una sesión activa, si el usuario sigue habilitado y si su rol le permite ejecutar la operación solicitada. La clase AuthGuard centraliza estas verificaciones, ofreciendo métodos para autenticación obligatoria y acceso exclusivo por rol. Además, se crea automáticamente un administrador inicial al arrancar la aplicación, garantizando que el sistema esté operativo desde el primer inicio.

4. Almacenamiento de datos

Para la persistencia se optó por H2 en modo servidor, proporcionando una base de datos ligera, fácil de distribuir e ideal para un entorno académico. Hibernate se encarga del mapeo entre las entidades del dominio y las tablas físicas. La capa de repositorios abstrae las operaciones sobre la base de datos, incluyendo la búsqueda de usuarios, la lectura y escritura de eventos, el control de inscripciones, el conteo de cupos y la recuperación de estadísticas. La clase TransactionManager organiza la ejecución de transacciones de lectura y escritura, desacoplando la lógica del negocio de los detalles de Hibernate.



La base de datos se crea automáticamente al ejecutarse la aplicación por primera vez, gracias a Flyway y un *script* de migración inicial. Este *script* define las tablas, llaves primarias, relaciones y restricciones necesarias. Este enfoque elimina pasos manuales de instalación, facilita la reproducibilidad, mantiene versionado el esquema de la base de datos y simplifica los despliegues en otros entornos.

La lógica centralizada para la creación, edición, publicación, cancelación y consulta de eventos se encuentra en la clase `EventService`. Las validaciones más importantes que se realizan son: asegurar una fecha futura al crear o editar, verificar que el cupo máximo sea mayor que cero, controlar los permisos según el rol, y validar que el nuevo cupo no sea menor que el número de inscritos actuales.

Por su parte, `RegistrationService` controla el proceso de inscripción. Evita inscripciones duplicadas e impide la inscripción en eventos no publicados, además de verificar la disponibilidad de cupos. También permite al participante cancelar su inscripción antes de la fecha del evento y reactivar una inscripción cancelada generando un nuevo *token* cuando sea necesario.

```
private void validate(UpsertEventCommand command) { 2 usages  Miguelrg28
    if (command.title() == null || command.title().isBlank()) {
        throw AppException.badRequest("El título es obligatorio.");
    }
    if (command.description() == null || command.description().isBlank()) {
        throw AppException.badRequest("La descripción es obligatoria.");
    }
    if (command.location() == null || command.location().isBlank()) {
        throw AppException.badRequest("El lugar es obligatorio.");
    }
    if (command.startsAt() == null) {
        throw AppException.badRequest("La fecha del evento es obligatoria.");
    }
    if (!command.startsAt().isAfter(LocalDate.now(clock))) {
        throw AppException.badRequest("La fecha del evento debe ser futura.");
    }
    if (command.maxCapacity() == null || command.maxCapacity() <= 0) {
        throw AppException.badRequest("El cupo máximo debe ser mayor que cero.");
    }
}
```

```
public Registration register(AuthenticatedUser actor, Long eventId) { Miguelrg28
    return transactionManager.write( Session session -> {
        Event event = eventRepository.findById(session, eventId)
            .orElseThrow(() -> AppException.notFound("Evento no encontrado."));

        validateRegistrationWindow(event);
        if (!event.isVisibleToParticipants()) {
            throw AppException.conflict("Solo los eventos publicados aceptan inscripciones.");
        }

        long activeRegistrations = registrationRepository.countActiveByEventId(session, eventId);
        if (activeRegistrations >= event.getMaxCapacity()) {
            throw AppException.conflict("El evento ya alcanzó su cupo máximo.");
        }

        User user = userRepository.findById(session, actor.id())
            .orElseThrow(() -> AppException.notFound("Usuario no encontrado."));

        Registration existing = registrationRepository.findByEventIdAndUserId(session, eventId, actor.id()).orElse(null);
        if (existing != null) {
            if (existing.isActive()) {
                throw AppException.conflict("Ya estás inscrito en este evento.");
            }
            existing.reactivate(newToken(), LocalDate.now(clock));
            return existing;
        }

        Registration registration = Registration.create(event, user, newToken(), LocalDate.now(clock));
        registrationRepository.save(session, registration);
        return registration;
    });
}
```

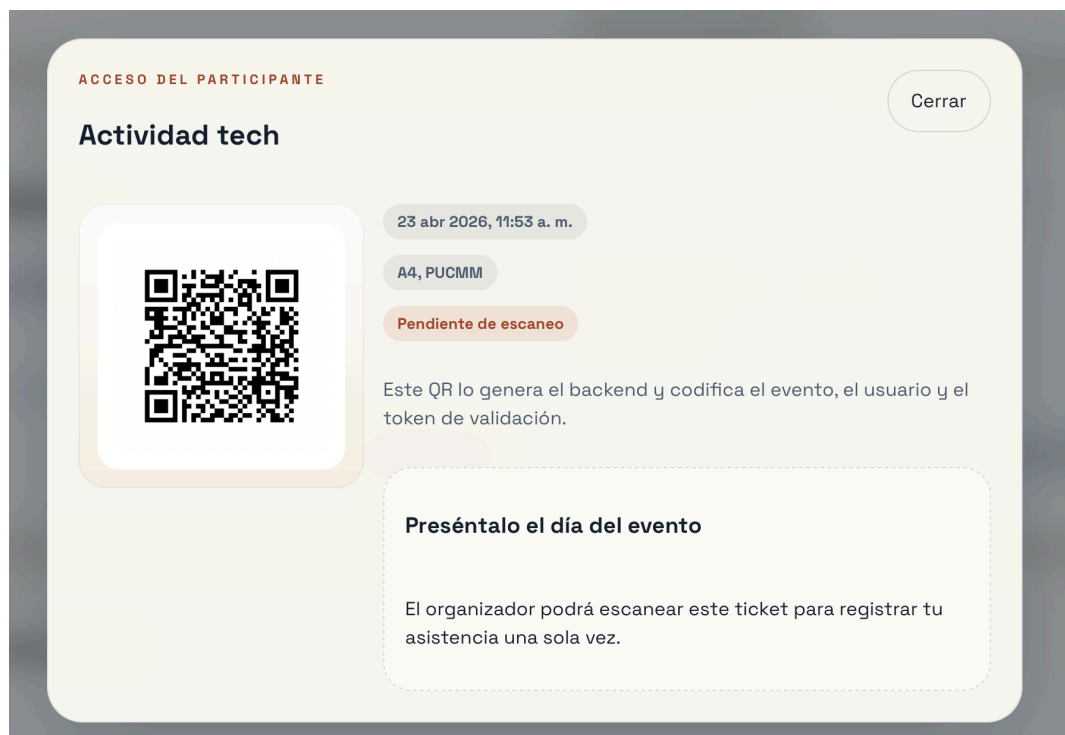
```
private void validateRegistrationWindow(Event event) { 1 usage  miguelrg28
    event.ensureNotCancelled();
    if (event.hasStarted(LocalDateTime.now(clock))) {
        throw AppException.conflict("No es posible inscribirse en un evento ya iniciado.");
    }
}
```

5. Generación y validación de QR

Cada inscripción activa genera un código QR único. El contenido del QR codifica el identificador del evento, el identificador del usuario y un token de validación. El backend utiliza ZXing para generar la imagen QR en formato PNG y para decodificar imágenes QR subidas desde la interfaz del organizador, permitiendo validar la asistencia tanto por cámara como por archivo de imagen.

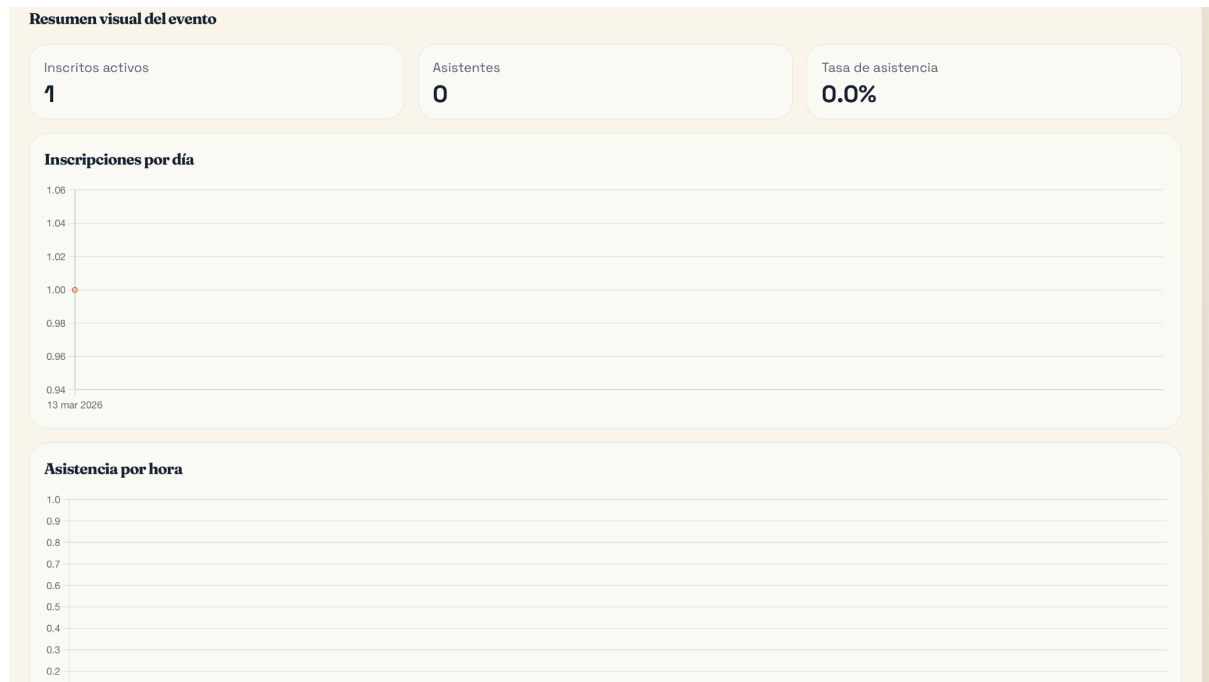
La lógica de asistencia se gestiona en AttendanceService, que verifica:

- que el evento corresponda al QR recibido,
- que la inscripción exista,
- que el QR no esté alterado,
- que no se marque asistencia dos veces,
- que la operación se realice el día del evento.



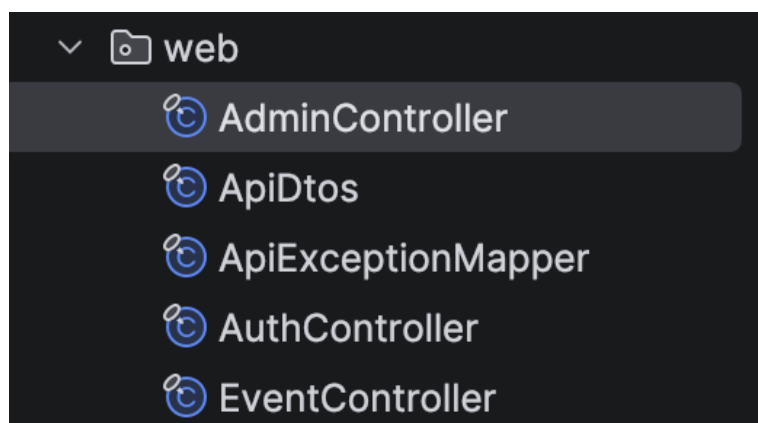
6. Estadísticas

El sistema incluye un módulo de estadísticas que calcula métricas clave, tales como el total de inscritos, el total de asistentes, el porcentaje de asistencia, las inscripciones por día y la asistencia por hora. Estas métricas son generadas en EventStatisticsService, agrupando las inscripciones por fecha de creación y la asistencia por hora, y se exponen a través de la API para que el *frontend* pueda renderizar gráficos dinámicos.



7. API Rest y conexión

La API está organizada por responsabilidad, incluyendo módulos para: autenticación, eventos e inscripciones, asistencia y QR, y administración. Los controladores principales son AuthController, EventController y AdminController. La respuesta de errores se estandariza mediante ApiExceptionHandler, garantizando que los errores se devuelvan de forma consistente en formato JSON.




```

83 export const apiClient = {
84   me: () => request('/api/auth/me'),
85   register: (payload) => request('/api/auth/register', { method: 'POST', body: payload }),
86   login: (payload) => request('/api/auth/login', { method: 'POST', body: payload }),
87   logout: () => request('/api/auth/logout', { method: 'POST' }),
88
89   listEvents: () => request('/api/events'),
90   listManagedEvents: () => request('/api/events/managed'),
91   listMyRegistrations: () => request('/api/my/registrations'),
92   createEvent: (payload) => request('/api/events', { method: 'POST', body: payload }),
93   updateEvent: (eventId, payload) => request('/api/events/${eventId}', { method: 'PUT', body: payload }),
94   publishEvent: (eventId) => request('/api/events/${eventId}/publish', { method: 'POST' }),
95   unpublishEvent: (eventId) => request('/api/events/${eventId}/unpublish', { method: 'POST' }),
96   cancelEvent: (eventId) => request('/api/events/${eventId}/cancel', { method: 'POST' }),
97   registerForEvent: (eventId) => request('/api/events/${eventId}/registrations', { method: 'POST' }),
98   cancelOwnRegistration: (eventId) => request('/api/events/${eventId}/registrations/me', { method: 'DELETE' }),
99   listEventRegistrations: (eventId) => request('/api/events/${eventId}/registrations'),
100   getEventSummary: (eventId) => request('/api/events/${eventId}/summary'),
101   scanAttendance: (eventId, qrContent) =>
102     request('/api/events/${eventId}/attendance/scan', {
103       method: 'POST',
104       body: { qrContent },
105     }),
106   scanAttendanceImage: (eventId, qrImage) => {
107     const body = new FormData()
108     body.append('qrImage', qrImage)
109
110     return request('/api/events/${eventId}/attendance/scan-image', {
111       method: 'POST',
112       body,
113     })
114   },
115   getQrMetadata: (eventId) => request('/api/events/${eventId}/registrations/me/qr'),
116   getQrImageUrl: (eventId) => buildUrl('/api/events/${eventId}/registrations/me/qr/image'),
117 }

```

7. Contenedor con Docker.

El proyecto incluye un Dockerfile multi-stage. La primera etapa compila el *frontend* con Node.js, la segunda empaqueta la aplicación Java con Gradle, y la última genera una imagen ligera con solo el *runtime* de Java. También se adjunta un docker-compose.yml para levantar la aplicación principal y un contenedor Nginx, con certificados TLS montados desde la carpeta deploy/certs. Nginx se configura como *proxy* inverso para redirigir automáticamente las solicitudes HTTP a HTTPS, cumpliendo con el requisito de seguridad y exposición pública.

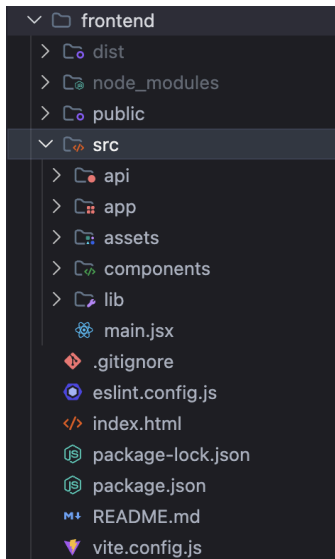
8. Frontend

Fue desarrollado con React 19 y Vite, utilizando la Fetch API para la comunicación. Para la visualización de datos se integró Chart.js y para la funcionalidad de escaneo se usó html5-qrcode. El diseño visual se complementó con CSS propio responsivo.

El *frontend* se organiza de la siguiente manera:

- **app**: composición principal de la aplicación.
- **components**: vistas y módulos funcionales.
- **api**: cliente HTTP centralizado.
- **lib**: utilidades de formato y reglas por rol.

La aplicación principal se define en `App.jsx`, donde se controlan la sesión del usuario, la carga inicial de datos, las notificaciones, el estado de las inscripciones, la gestión de eventos administrados, el resumen estadístico y la visualización del *ticket* QR.

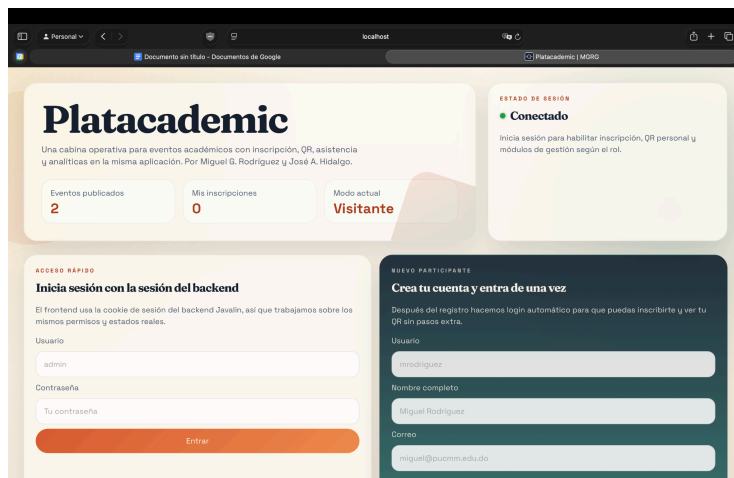


El archivo `client.js` centraliza todas las peticiones a la API, evitando la duplicación de URLs y manteniendo una única forma de enviar formularios, manejar errores, compartir la sesión con credentials: include, y consumir los *endpoints* de autenticación, eventos, administración, QR y asistencia. Este diseño permite que el *frontend* y el *backend* trabajen con la misma sesión real del usuario.

La interfaz de usuario está compuesta por varios módulos clave:

- **AuthPanels**: Permite iniciar sesión, registrar nuevos participantes y mostrar errores del proceso de autenticación. Tras el registro, se ejecuta un *login* automático para mejorar la experiencia.
- **EventCatalog**: Muestra los eventos publicados en formato de tarjetas y ofrece funcionalidades de búsqueda por texto, inscripción, cancelación de inscripción y acceso al QR del participante.
- **ParticipantWorkspace**: Presenta las inscripciones activas del usuario y proporciona acceso rápido al *ticket* QR.
- **QrTicketModal**: Muestra el QR generado por el *backend* junto con la información del evento y el estado de asistencia.

- **ManagerWorkspace:** Es el centro operativo para el organizador, incluyendo el formulario de creación y edición, la lista de eventos gestionados, el panel de operaciones, el lector de QR, la tabla de inscritos y una sección analítica.
- **QrScannerPanel:** Módulo para la validación de asistencia, permitiendo el escaneo tanto por cámara como por carga de imagen QR.
- **SummaryCharts:** Renderiza las métricas del evento mediante gráficos de líneas y barras.
- **AdminWorkspace:** Proporciona al administrador capacidades para búsqueda global, bloqueo y desbloqueo de usuarios, asignación o revocación del rol de organizador, y eliminación de eventos inapropiados.



Utiliza una interfaz responsiva basada en CSS propio. Se implementaron los siguientes elementos: distribución tipo grid, tarjetas para eventos y métricas, un modal para el ticket QR, estilos diferenciados para paneles según el rol, adaptación a dispositivos móviles, y feedback visual para operaciones exitosas o fallidas. La interfaz se diseñó para mantener la claridad visual y la separación de funciones, asegurando que cada usuario vea solo los módulos que necesita.

Conclusión

El proyecto desarrollado cumple con los requerimientos fundamentales del Sistema de Gestión y Control de Eventos Académicos y demuestra la integración efectiva de tecnologías modernas tanto del lado del servidor como del cliente. La solución implementa autenticación por sesión, control de roles, gestión de eventos, control de inscripciones, generación y lectura de QR, estadísticas visuales y despliegue mediante Docker.

Desde el punto de vista académico, este trabajo permitió aplicar conocimientos de modelado de objetos, desarrollo web full stack, persistencia con ORM, interacción asíncrona con Fetch API, diseño responsivo y despliegue seguro. Desde el punto de vista técnico, el

sistema queda organizado por capas, con separación clara de responsabilidades y preparado para seguir evolucionando.

Como mejoras futuras, podrían incorporarse notificaciones por correo, exportación de reportes, filtros avanzados, dashboards históricos y una mayor automatización del despliegue en producción.

Anexos

1. [Entrar al repositorio](#)
2. Ver vídeo explicando todo.